MANUAL TECNICO

Proyecto Uno-Fase Dos Compiladores 2 : C3D-Compiler
Version 1.0

Desarrollador: Ronaldo Hilario.

Tecnologias Utilizada

Frontend: Angular version 21.
Backend: Java version 21.
Generador Lexer Y Parser: Antlr4.
Protocolo de Comunicación: API REST

Diagrama De Clases:

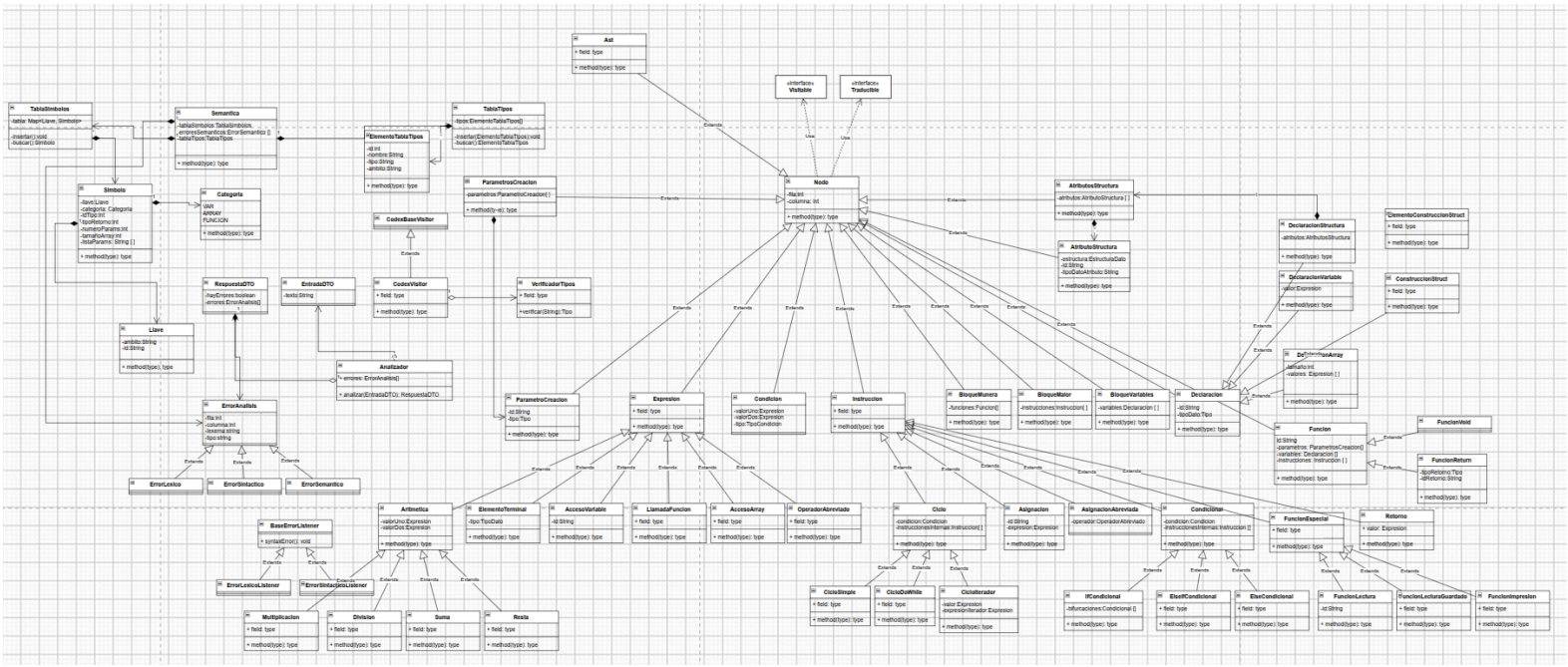


TABLA COMPATIBILIDAD

BOOL = bool, boolean
LITTERA = litera, cadena, string
NUMERUS = numerus, entero, int
DECIMALIS = decimalis, flotante, double
TEXTUM = textum, caracter, char

Contatenacion (+)

	BOOL	LITTERA	NUMERUS	DECIMALIS	TEXTUM
BOOL	error	error	error	error	textum
LITTERA	error	numerus	numerus	decimalis	textum
NUMERUS	error	numerus	numerus	decimalis	textum
DECIMALIS	error	decimalis	decimalis	decimalis	textum
TEXTUM	textum	textum	textum	textum	textum

Otras Operaciones(-, *, /)

	BOOL	LITTERA	NUMERUS	DECIMALIS	TEXTUM
BOOL	error	error	error	error	error
LITTERA	error	numerus	numerus	decimalis	error
NUMERUS	error	numerus	numerus	decimalis	error
DECIMALIS	error	decimalis	decimalis	decimalis	error
TEXTUM	error	error	error	error	error

GRAMATICAS

Gramatica Y

```
grammar LenguajeY;

tokens { INDENT, DEDENT }

@lexer::header {
import com.yuvalshavit.antlr4.DenterHelper;
}

@lexer::members {
   private final DenterHelper denter = DenterHelper.builder()
       .nl(NL)
       .indent(LenguajeYParser.INDENT)
       .dedent(LenguajeYParser.DEDENT)
       .pullToken(LenguajeYLexer.super::nextToken);

   @Override
   public Token nextToken() {
      return denter.nextToken();
   }
}

/** ANALISIS SINTACTICO **/

lenguaje
   : NL* bloque_estructuras? NL* bloque_funciones? NL* EOF
   ;

/**BLOQUE DE ESTRUCTURAS**/
bloque_estructuras
   : ESTRUCTURAS NL+ estructura+
   ;

estructura
   : ESTRUCTURA ID_STRUCTURA DOS_P INDENT atributo_struct+ DEDENT NL*
   ;

atributo_struct
   : tipo_dato_general ID (CORCH_A NUM_ENTERO CORCH_C)? NL+
```

```
    ;

tipo_dato_general
    : tipo_dato_primitivo
    | ID_STRUCTURA
    ;

/**BLOQUE DE FUNCIONES**/
bloque_funciones
    : FUNCIONES NL+ funcion+
    ;

funcion
    : funcion_void NL*
    | funcion_retorno NL*
    ;

funcion_void
    : DEFINIR ID PAR_A params? PAR_C DOS_P bloque
    ;

funcion_retorno
    : DEFINIR ID PAR_A params? PAR_C FLECHA tipo_dato_general DOS_P bloque
    ;

params
    : param (COMA param)*
    ;

param
    : tipo_dato_primitivo ID
    | CORCH_A CORCH_C tipo_dato_primitivo ID
    | LLAVE_A LLAVE_C ID_STRUCTURA ID
    ;

/** INSTRUCCIONES Y BLOQUES **/

bloque
    : INDENT instruccion+ DEDENT
    ;

instruccion
    : ( declaracion
      | fun_leer
```

```
      | asignacion
      | suma_resta_abrev
      | fun_imprimir
      | llamada_funcion
      | ROMPER
      | CONTINUAR
      | retorno
      ) NL+
    | inst_si
    | inst_elegir
    | ciclo_para
    | ciclo_mientras
    | ciclo_hacer NL+
    | NL
    ;


/**DECLARACION DE VARIABLES: SIMPLES, ARRAYS Y ESTRUCTURAS**/
declaracion
    : dec_var_simple
    | dec_array
    | dec_estruct
    ;

dec_var_simple
    : tipo_dato_primitivo ID (EQ expresion)?
    ;

dec_array
    : tipo_dato_primitivo ID (CORCH_A expresion CORCH_C)+ (EQ LLAVE_A valores_iniciales?
LLAVE_C)?
    ;

dec_estruct
    : ID_STRUCTURA ID ( (EQ LLAVE_A valores_iniciales? LLAVE_C)? | (EQ expresion)? )
    ;

/** ASIGNACION Y L-VALUES **/

lvalue
    : ID
    | lvalue CORCH_A expresion CORCH_C
    | lvalue PUNTO ID
    ;
```

```
asignacion
   : lvalue EQ expresion
   ;

valores_iniciales
   : expresion (COMA expresion)*
   ;

/**CONDICIONAL SI**/
inst_si
   : SI PAR_A expresion PAR_C ENTONCES bloque bifurcacion?
   ;

bifurcacion
   : SINO PAR_A expresion PAR_C ENTONCES bloque bifurcacion?
   | CONTRARIO bloque
   ;

/**ELEGIR (SWITCH) **/
inst_elegir
   : ELEGIR PAR_A expresion PAR_C DOS_P casos
   ;

casos
   : INDENT caso+ DEDENT
   ;

caso
   : CASO expresion DOS_P bloque
   | SIEMPRE DOS_P bloque
   ;

/**CICLOS**/

/**PARA (FOR)**/
ciclo_para
   : PARA PAR_A dec_iterador P_COMA expresion P_COMA (suma_resta_abrev | asignacion) PAR_C
DOS_P bloque
   ;

dec_iterador
   : tipo_dato_primitivo ID EQ expresion
   | ID EQ expresion
```

```
    ;

/**MIENTRAS (WHILE)**/
ciclo_mientras
    : MIENTRAS PAR_A expresion PAR_C HACER bloque
    ;

/**HACER MIENTRAS (DO-WHILE)**/
ciclo_hacer
    : HACER DOS_P bloque MIENTRAS PAR_A expresion PAR_C
    ;

/**FUNCIONES ESPECIALES**/
fun_imprimir
    : IMPRIMIR PAR_A expresion PAR_C
    ;

fun_leer
    : LEER PAR_A PAR_C
    ;

llamada_funcion
    : ID PAR_A argumentos? PAR_C
    ;

argumentos
    : expresion (COMA expresion)*
    ;

suma_resta_abrev
    : lvalue MAS_MAS
    | lvalue MENOS_MENOS
    ;

retorno
    : RETORNAR expresion?
    ;


/**GENERALES**/
expresion
    : PAR_A expresion PAR_C                 # expParentesis
    | llamada_funcion                       # expLlamada
    | expresion PUNTO ID                    # expAcceso
```

```
    | expresion CORCH_A expresion CORCH_C          # expIndice
    | fun_leer                                    # expFunLeer
    | NOT expresion                               # expNot
    | MENOS expresion                             # expNegativo
    | expresion (MULTI | DIV) expresion           # expMultDiv
    | expresion (MAS | MENOS) expresion           # expSuma
    | expresion (MENOR_Q | MENOR_EQ_Q
            | MAYOR_Q | MAYOR_EQ_Q) expresion     # expRelacional
    | expresion (EQ_EQ | NO_EQ) expresion         # expIgualdad
    | expresion AND expresion                     # expAnd
    | expresion OR expresion                      # expOr
    | NUM_ENTERO                                  # expEntero
    | NUM_DECIMAL                                 # expDecimal
    | LIT_CADENA                                  # expCadena
    | CHAR                                        # expChar
    | VERDADERO                                   # expVerdadero
    | FALSO                                       # expFalso
    | ID                                          # expId
    ;

tipo_dato_primitivo
    : ENTERO
    | FLOTANTE
    | CADENA
    | CARACTER
    | BOOL
    ;



/**ANALISIS LEXICO**/

/**Palabras reservadas**/
ESTRUCTURAS: '%estructuras';
FUNCIONES: '%funciones';
ESTRUCTURA: 'estructura';
ENTERO: 'entero';
CADENA: 'cadena';
FLOTANTE: 'flotante';
CARACTER: 'caracter';
BOOL: 'bool';
VERDADERO: 'verdadero';
FALSO: 'falso';
DEFINIR: 'definir';
```

```
RETORNAR: 'retornar';
SINO: 'sino';
SI: 'si';
CONTRARIO: 'contrario';
ENTONCES: 'entonces';
IMPRIMIR: 'imprimir';
ELEGIR: 'elegir';
CASO: 'caso';
SIEMPRE: 'siempre';
ROMPER: 'romper';
PARA: 'para';
CONTINUAR: 'continuar';
MIENTRAS: 'mientras';
HACER: 'hacer';
LEER: 'leer';

/**simbolos**/
FLECHA: '->';
MAS:    '+';
MENOS:  '-';
MULTI:  '*';
DIV:    '/';
EQ: '=';
EQ_EQ:  '==';
NO_EQ:  '!=';
MAYOR_Q: '>';
MAYOR_EQ_Q: '>=';
MENOR_Q: '<';
MENOR_EQ_Q: '<=';
AND:    '&&';
OR:     '||';
NOT:    '!';
MAS_MAS:    '++';
MENOS_MENOS: '--';
PUNTO:  '.';
COMA:   ',';
DOS_P:  ':';
P_COMA: ';';
LLAVE_A:    '{';
LLAVE_C:    '}';
CORCH_A:    '[';
CORCH_C:    ']';
PAR_A:  '(';
PAR_C:  ')';
```

```
/**Expresiones regulares**/
ID_STRUCTURA: [A-Z][a-zA-Z0-9_]* ;
ID: [a-zA-Z_][a-zA-Z0-9_]* ;
NUM_ENTERO: [0-9]+ ;
NUM_DECIMAL: [0-9]+ '.' [0-9]+;
LIT_CADENA : "" .*? "" ;
CHAR : '\" . '\" ;


NL          : ('\r'? '\n' | '\r') [ \t\u00A0]* ;
WS          : [ \t\u00A0]+ -> skip ;
COMENTARIO_LINEA  : '//' ~[\r\n]* -> channel(HIDDEN) ;
COMENTARIO_BLOQUE : '/*' .*? '*/' -> channel(HIDDEN) ;
```

Gramatica Z

```
grammar LenguajeZ;


/**ANALISIS SINTACTICO**/
programa
    : clase EOF
    ;

clase
    : (PUBLIC)? CLASS ID (EXTENDS ID)? LLAVE_A contenido LLAVE_C
    ;

contenido
    : (atributo | metodo | constructor)*
    ;


/**ATRIBUTO**/
atributo
    : (mod_acceso)? declaracion P_COMA
    ;

declaracion
    : dec_array
    | dec_var_simple
    ;

dec_var_simple
    : tipo_dato_general ID (EQ expresion)?
    ;

dec_array
    : tipo_dato_general
    (CORCH_A expresion? CORCH_C)+
    ID
    (EQ (
            LLAVE_A valores_iniciales? LLAVE_C
        |   NEW tipo_dato_general (CORCH_A expresion CORCH_C)+
        |   expresion
        )
    )?
    ;
```

```
valores_iniciales
   : expresion (COMA expresion)*
   ;

/**CONSTRUCTOR**/
constructor
   : (mod_acceso)? ID PAR_A parametros? PAR_C LLAVE_A llamada_super_constructor? instruccion*
LLAVE_C
   ;


instruccion
   :
   (declaracion
   | asignacion
   | suma_resta_abrev
   | llamada_metodo
   | llamada_this
   | llamada_super
   | llamada_metodo_objeto
   | funcion_especial
   | BREAK
   | CONTINUE
   | return
   ) P_COMA+
   | inst_if
   | inst_switch
   | ciclo_for
   | ciclo_while
   | ciclo_do_while
   ;

asignacion
   : lvalue (EQ | MAS_EQ | MENOS_EQ | MULTI_EQ) expresion
   ;

lvalue
   : ID
   | llamada_this
   | llamada_super
   | lvalue CORCH_A expresion CORCH_C
   | lvalue PUNTO ID
   ;
```

```
return
   : RETURN (expresion)?
   ;


llamada_metodo_objeto
   : expresion PUNTO llamada_metodo
   ;

/**CONDICIONALES**/
inst_if
   : IF PAR_A expresion PAR_C cuerpo_if bifurcacion?
   ;

cuerpo_if
   : instruccion
   | LLAVE_A instruccion* LLAVE_C
   ;

bifurcacion
   : ELSE IF PAR_A expresion PAR_C cuerpo_if bifurcacion?
   | ELSE cuerpo_if
   ;


/**SWITCH**/
inst_switch
   : SWITCH PAR_A expresion PAR_C LLAVE_A casos? LLAVE_C
   ;

casos
   : CASE expresion DOS_P instruccion* casos?
   | DEFAULT DOS_P instruccion*
   ;



/**CICLOS**/
ciclo_for
   :
   FOR PAR_A
      dec_iterador? P_COMA
      expresion? P_COMA
```

```
      (suma_resta_abrev | asignacion)?
   PAR_C LLAVE_A instruccion* LLAVE_C
   ;

dec_iterador
   : declaracion
   | asignacion
   ;

ciclo_while
   : WHILE PAR_A expresion PAR_C LLAVE_A instruccion* LLAVE_C
   ;

ciclo_do_while
   : DO LLAVE_A instruccion* LLAVE_C WHILE PAR_A expresion PAR_C P_COMA
   ;

/**METODO**/
metodo
   : funcion
   | procedimiento
   ;

/**void**/
procedimiento
   : OVERRIDE? (mod_acceso)? VOID ID PAR_A parametros? PAR_C LLAVE_A instruccion*
LLAVE_C
   ;

funcion
   : OVERRIDE? (mod_acceso)? tipo_dato_general (CORCH_A CORCH_C)* ID PAR_A parametros?
PAR_C LLAVE_A instruccion* LLAVE_C
   ;

parametros
   : parametro (COMA parametro)*
   ;

parametro
   : tipo_dato_general (CORCH_A  CORCH_C)* ID
   ;

llamada_metodo
   : ID PAR_A argumentos? PAR_C
```

```
    ;

argumentos
    : expresion (COMA expresion)*
    ;


/**FUNCIONES ESPECIALES**/

funcion_especial
    : fun_leer
    | fun_imprimir_con_ln
    | fun_imprimir_sin_ln
    ;

fun_leer
    : READLN PAR_A expresion? PAR_C
    ;

fun_imprimir_con_ln
    : PRINTLN PAR_A expresion? PAR_C
    ;

fun_imprimir_sin_ln
    : PRINT PAR_A expresion PAR_C
    ;

suma_resta_abrev
    : lvalue MAS_MAS
    | lvalue MENOS_MENOS
    ;


expresion
    : expr_base                                     # expBase
    | expr_base INTERROGACION expresion DOS_P expresion     # expTernaria
    ;

expr_base
    : PAR_A expresion PAR_C                         # expParentesis
    | llamada_this                                  # expThis
    | llamada_super                                 # expSuper
    | llamada_metodo                                # expLlamada
    | expr_base PUNTO ID                            # expAcceso
```

```
    | expr_base PUNTO llamada_metodo                    # expLlamadaEncadenada
    | expr_base CORCH_A expresion CORCH_C                 # expIndice
    | expr_base MAS_MAS                                   # expPostIncremento
    | expr_base MENOS_MENOS                               # expPostDecremento
    | fun_leer                                            # expFunLeer
    | MAS_MAS expr_base                                   # expPreIncremento
    | MENOS_MENOS expr_base                               # expPreDecremento
    | NOT expr_base                                       # expNot
    | MENOS expr_base                                     # expNegativo
    | MAS expr_base                                       # expPositivo
    | expr_base (MULTI | DIV | MODULO) expr_base          # expMultDivMod
    | expr_base (MAS | MENOS) expr_base                   # expSumaResta
    | expr_base (MENOR_Q | MENOR_EQ_Q
            | MAYOR_Q | MAYOR_EQ_Q) expr_base             # expRelacional
    | expr_base (EQ_EQ | NO_EQ) expr_base                 # expIgualdad
    | expr_base AND expr_base                             # expAnd
    | expr_base OR expr_base                              # expOr
    | NEW tipo_dato_general PAR_A argumentos? PAR_C       # expNewObjeto
    | NEW tipo_dato_general (CORCH_A expresion CORCH_C)+  # expNewArreglo
    | ENTERO                                              # expEntero
    | DECIMAL                                             # expDecimal
    | CADENA                                              # expCadena
    | LIT_CHAR                                            # expChar
    | TRUE                                                # expVerdadero
    | FALSE                                               # expFalso
    | ID                                                  # expId
    | NULL                                                # expNull
    ;


tipo_dato_general
    : tipo_dato_primitivo
    | ID
    ;

tipo_dato_primitivo
    : INT
    | DOUBLE
    | STRING
    | CHAR
    | BOOLEAN
    ;
```

```
mod_acceso
   : PUBLIC
   | PRIVATE
   | PROTECTED
   ;

//llamada a atributos o metodos this.atributo this.metodo(), this.metodo(argumento)
llamada_this
   : THIS PUNTO ( ID | llamada_metodo)
   ;

//llamada a atributos o metodos super.atributo super.metodo(), super.metodo(argumento)
llamada_super
   : SUPER PUNTO ( ID | llamada_metodo)
   ;

//llamada al constructor de la superclase, solo puede ser la primera instruccion del constructor
llamada_super_constructor
   : SUPER PAR_A argumentos? PAR_C P_COMA+
   ;

/**ANALISIS LEXICO**/

/**Palabras Reservadas**/
PUBLIC:     'public';
PRIVATE:    'private';
PROTECTED:  'protected';
EXTENDS:    'extends';
OVERRIDE:   '@Override';
THIS:       'this';
SUPER:      'super';
CLASS:      'class';
VOID:       'void';
INT:        'int';
DOUBLE:     'double';
STRING:     'String';
CHAR:       'char';
BOOLEAN:    'boolean';
NEW:        'new';
RETURN:     'return';
IF:         'if';
ELSE:       'else';
TRUE:       'true';
FALSE:      'false';
```

```
SWITCH:     'switch';
CASE:       'case';
BREAK:      'break';
CONTINUE:   'continue';
DEFAULT:    'default';
PRINTLN:    'println';
PRINT:      'print';
READLN:     'readln';
FOR:        'for';
WHILE:      'while';
DO:         'do';
NULL:       'null';

/**Simbolos**/
MAS_EQ:     '+=';
MENOS_EQ:   '-=';
MULTI_EQ:   '*=';
MAS:        '+';
MENOS:      '-';
MULTI:      '*';
DIV:        '/';
MODULO:     '%';
EQ:         '=';
EQ_EQ:      '==';
NO_EQ:      '!=';
MAYOR_Q:    '>';
MAYOR_EQ_Q: '>=';
MENOR_Q:    '<';
MENOR_EQ_Q: '<=';
AND:        '&&';
OR:         '||';
NOT:        '!';
MAS_MAS:    '++';
MENOS_MENOS:'--';
PUNTO:      '.';
COMA:       ',';
DOS_P:      ':';
P_COMA:     ';';
LLAVE_A:    '{';
LLAVE_C:    '}';
CORCH_A:    '[';
CORCH_C:    ']';
PAR_A:      '(';
PAR_C:      ')';
```

```
INTERROGACION: '?';


/**expresiones regulares**/

ID:         [a-zA-Z_][a-zA-Z0-9_]* ;
ENTERO:     [0-9]+ ;
DECIMAL:    [0-9]+ '.' [0-9]+;
CADENA :    "" .*? "" ;
LIT_CHAR :  '\" . '\" ;

WS: [ \t\n\r\f\u00A0\u200B] -> skip;
COMENTARIO_LINEA  : '//' ~[\r\n]* -> channel(HIDDEN) ;
COMENTARIO_BLOQUE : '/*' .*? '*/' -> channel(HIDDEN) ;
```

Gramatica Pig

```
grammar LenguajePig;

/** ANALISIS SINTACTICO **/

pig
   : importacion* bloque_variabile? bloque_maior
   ;

importacion
   : IMPORT importe P_COMA?
   ;

importe
   : ID (PUNTO ID)*
   ;

bloque_variabile
   : VARIABILES declaracion*
   ;

declaracion
   : ( dec_var
     | asignacion
     ) P_COMA+
   ;

// Declaraciones de variables, estructuras u objetos
dec_var
   : ESTO ID (DOS_P (tipo_dato | ID)? expresion)? (DOS_P ID)?
   | SERIES ID CORCH_A expresion CORCH_C DOS_P? (tipo_dato | ID)? expresion?
   ;

tipo_dato
   : NUMERUS
   | DECIMALIS
   | TEXTUM
   | LITTERA
   | BOOL
   ;

// Asignacion
asignacion
```

```
    : lvalue EQ expresion
    ;

lvalue
    : ID
    | lvalue CORCH_A expresion CORCH_C
    | lvalue PUNTO ID
    ;

argumentos
    : expresion (COMA expresion)*
    ;

instanciacion
    : NOVUS ID PAR_A argumentos? PAR_C
    ;

bloque_maior
    : MAIOR instruccion* (FINIS | FINIS_MAY) P_COMA?
    ;

instruccion
    : condicional
    | ciclo_simple
    | ciclo_do_while
    | ciclo_iterador
    | operacion_abrev P_COMA?
    | declaracion
    | asignacion P_COMA?
    | fun_lectura
    | fun_lectura_guardado
    | expresion P_COMA?
    | fun_impresion
    | PERGE P_COMA?
    | INTERRUMPE P_COMA?
    ;


/** CONDICIONALES **/

condicional
    : SI PAR_A expresion PAR_C LLAVE_A instruccion* LLAVE_C
      rama_aliter*
      (FINIS | FINIS_MAY) P_COMA?
```

```
    ;

rama_aliter
    : ALITER PAR_A expresion PAR_C LLAVE_A instruccion* LLAVE_C
    | ALITER LLAVE_A instruccion* LLAVE_C
    ;

/** CICLOS **/

ciclo_simple
    : DUM PAR_A expresion PAR_C LLAVE_A instruccion* LLAVE_C (FINIS | FINIS_MAY)
P_COMA?
    ;

ciclo_do_while
    : FACERE LLAVE_A instruccion* LLAVE_C DUM PAR_A expresion PAR_C P_COMA?
    ;

ciclo_iterador
    : PER PAR_A dec_var_sin_pcoma P_COMA expresion P_COMA expresion_iterador PAR_C
LLAVE_A instruccion* LLAVE_C
    ;

dec_var_sin_pcoma
    : ESTO ID DOS_P? (tipo_dato | ID)? expresion?
    ;

expresion_iterador
    : expresion
    | operacion_abrev
    | asignacion
    ;

operacion_abrev
    : ID MAS_MAS
    | ID MENOS_MENOS
    ;

llamada_metodo
    : ID PAR_A argumentos? PAR_C
    ;

// Lectura por consola
fun_lectura
```

```
    : MENOR_Q MENOR_Q P_COMA?
    ;

fun_lectura_guardado
    : lvalue MENOR_Q MENOR_Q P_COMA?
    ;

// Impresion en consola encadenada
fun_impresion
    : (MAYOR_Q MAYOR_Q expresion)+ P_COMA?
    ;

/** EXPRESIONES **/

expresion
    : PAR_A expresion PAR_C                     # expParentesis
    | llamada_metodo                            # expLlamada
    | expresion PUNTO llamada_metodo            # expLlamadaMetodo
    | expresion PUNTO ID                        # expAcceso
    | expresion CORCH_A expresion CORCH_C       # expIndice
    | instanciacion                             # expInstanciacion
    | LLAVE_A argumentos? LLAVE_C               # expArgStruct
    | NON expresion                             # expNot
    | MENOS expresion                           # expNegativo
    | expresion (MULTI | DIV) expresion         # expMultiDiv
    | expresion (MAS | MENOS) expresion         # expSumaResta
    | expresion (MENOR_Q | MENOR_EQ_Q
            | MAYOR_Q | MAYOR_EQ_Q) expresion   # expRelacional
    | expresion (EQ_EQ | NO_EQ) expresion       # expIgualdad
    | expresion AND expresion                   # expAnd
    | expresion OR expresion                    # expOr
    | ENTERO                                    # expEntero
    | DECIMAL                                   # expDecimal
    | CADENA                                    # expCadena
    | CHAR                                      # expChar
    | VERUM                                     # expVerdadero
    | FALSUS                                    # expFalso
    | ID                                        # expId
    | NULL                                      #expNull
    ;

/** ANALISIS LEXICO **/

/** Palabras reservadas **/
```

```
VARIABILES: 'VARIABILES>';
MAIOR:      'MAIOR>';
IMPORT:     'import';
ESTO:       'esto';
SERIES:     'series';
NOVUS:      'novus';
NUMERUS:    'numerus';
DECIMALIS:  'decimalis';
TEXTUM:     'textum';
LITTERA:    'littera';
BOOL:       'bool';
VERUM:      'verum';
FALSUS:     'falsus';
SI:         'si';
FINIS:      'finis';
FINIS_MAY:  'FINIS';
ALITER:     'aliter';
DUM:        'dum';
FACERE:     'facere';
PER:        'per';
PERGE:      'perge';
INTERRUMPE: 'interrumpe';
NON:        'non';
NULL:       'null';
/** Simbolos **/
MAS:        '+';
MENOS:       '-';
MULTI:      '*';
DIV:        '/';
EQ:         '=';
EQ_EQ:      '==';
NO_EQ:      '!=';
MAYOR_Q:    '>';
MAYOR_EQ_Q:  '>=';
MENOR_Q:    '<';
MENOR_EQ_Q:  '<=';
AND:        '&&';
OR:         '||';
MAS_MAS:    '++';
MENOS_MENOS: '--';
PUNTO:      '.';
COMA:       ',';
DOS_P:      ':';
P_COMA:     ';';
```

```
LLAVE_A:     '{';
LLAVE_C:     '}';
CORCH_A:     '[';
CORCH_C:     ']';
PAR_A:       '(';
PAR_C:       ')';

// Expresiones regulares
ID:              [a-zA-Z_][a-zA-Z0-9_]* ;
ENTERO:          [0-9]+ ;
DECIMAL:          [0-9]+ '.' [0-9]+ ;
CADENA:           "" .*? "" ;
CHAR:            '\" . '\" ;
COMENTARIO_LINEA:  '//' ~[\r\n]* -> channel(HIDDEN) ;
COMENTARIO_BLOQUE: '##' .*? '##' -> channel(HIDDEN) ;
WS:              [ \t\n\r\f\u00A0\u200B]+ -> skip ;
```